UCPERT SDP

UCPERT is a web-based application developed to simplify university students course registration processes, automate graduation audits, and intelligently recommend elective courses

1. Problem and Project Context

- **What problem are we solving?** The difficulties students face when tracking graduation requirements, avoiding schedule conflicts, navigating complex prerequisite structures, and choosing elective courses

- **Who experiences the problem?** Undergraduate and associate degree students, academic advisors, and departmental secretaries

- **What evidence or observation supports the problem?** Incorrect course registrations leading to delayed graduations, high workloads during advising hours, and students being unaware of which courses to select as elective

- **What is the intended outcome for users or an organization?** Enabling students to create error-free, optimized course schedules, automating degree audits, and providing smart, career-oriented elective recommendations

2. Stakeholders and Users

Students:

- **Goals:** Build course schedules quickly, view graduation status, and select the right electives
- **Constraints:** Credit limits, time conflicts, mandatory prerequisites

Academic Advisors:

- **Goals:** Quickly approve student schedules and reduce advising workloads
- **Constraints:** Time constraints and the difficulty of reviewing thousands of students individually

Possible Conflicts: Desired elective courses reaching full capacity or advisor rejection of schedules due to time conflicts

3. Scope

In scope for the first release: User registration/login, curriculum upload (JSON/PDF based), schedule conflict detection, prerequisite validation, and a basic rule-based elective recommendation engine

Explicitly out of scope: Direct live integration with official university automation systems (manual course list uploads will be used initially)

Three initial success criteria:

- 100% accurate detection of time conflicts in student schedules
- Generating a course schedule in under 2 minutes
- At least 80% of users finding recommended electives aligned with their major

Major assumptions and dependencies: Departmental curriculum data is assumed to be up-to-date and accurate.

4. Initial Process and Work Plan

Process model: 2-week sprints.

Team roles and responsibilities: Backend Developer, Frontend Developer, QA/Testing Engineer

Initial milestones through Week 12:

- **Week 4:** Requirements analysis and SDP submission
- **Week 6:** Database design and core API integrations
- **Week 8:** UI prototypes and schedule conflict algorithm
- **Week 10:** Elective recommendation module and integration tests
- **Week 12:** End-user testing, bug fixing, and final submission

Communication and decision-making practices: Communication via project Whattsapp group, weekly physical/online meetings; task tracking via github

5. Technology and Architecture Direction

Technology Stack:

- **Frontend:** Next.js (For rapid development and modern UI components)
- **Backend:** Python / FastAPI (For swift API development and recommendation algorithm compatibility)
- **Database:** PostgreSQL (To securely store relational curriculum, student, and course data)

Quality Attributes:

- **Performance:** Schedule conflict queries responding under 200 ms
- **Usability:** Mobile and desktop responsive design
- **Reliability:** Security and encryption of student data

Initial context or system-boundary diagram: Student -> Web Interface (Frontend) -> REST API (Backend) -> PostgreSQL (Database)

Potential Drawbacks:

- Choosing python as the backend language might not get us to the previously chosen under 2 min. recommedation engine
- PostgreSQL might be an overkill for the initial release

6. User Experience or Interaction Evidence

A possible design idea for student interaction is given in "design_idea.pdf"

7. Repository and Collaboration Setup

GitHub Repository URL: https://github.com/ayldzli/UCPERT

Branch and pull-request workflow: GitFlow model (main, dev, feature/* branches with mandatory pull requests)

8. Initial Risk Register

- **Risk:** Irregular curriculum data formats. (Likelihood: High, Impact: High)
- **Mitigation:** Creating a standardized JSON template
- **Risk:** Slow execution of the recommendation algorithm. (Likelihood: Medium, Impact: Medium)
- **Mitigation:** Implementing data caching mechanisms
- **Risk:** Team member falling behind on workload. (Likelihood: Low, Impact: High)
- **Mitigation:** Early scheduling of tasks and flexible backup planning
- **Risk:** Logic bugs in the schedule conflict algorithm. (Likelihood: Medium, Impact: High)
- **Mitigation:** Writing comprehensive unit tests
- **Risk:** UI breaking on mobile devices. (Likelihood: Medium, Impact: Low)
- **Mitigation:** Potentially adopting a mobile-first approach using Tailwind CSS

9. Responsible AI-Use Plan

- **Which activities may use them:** Code scaffolding, code generation, algorithm optimization, and documentation assistance
- **How generated code or text will be reviewed and tested:** All generated code will be manually reviewed and tested by team members
- **How dependencies, licenses, and sensitive data will be checked:** Open-source library licenses will be reviewed, and no sensitive student data will be shared with AI tools
- **How important AI-assisted decisions will be documented:** Key architectural decisions influenced by AI will be recorded in design documents